

GUIA PARA ESTRUTURAR O HARNESS DE SOFTWARE

Como documentar seu projeto para IA e times de desenvolvimento

ÍNDICE

- 1. [O que é um Harness?](#)
- 2. [Estrutura dos 4 Documentos](#)
- 3. [PLAN.md - Arquitetura do Sistema](#)
- 4. [SPEC.md - Especificação Funcional](#)
- 5. [HOOKS.md - Fluxos Detalhados](#)
- 6. [AGENTS.md - Setup para IA](#)
- 7. [Checklist de Criação](#)
- 8. [Templates Prontos](#)

1. O QUE É UM HARNESS?

Um **Harness** é um conjunto de 4 documentos que descrevem completa e estruturadamente um projeto de software. Ele serve para:

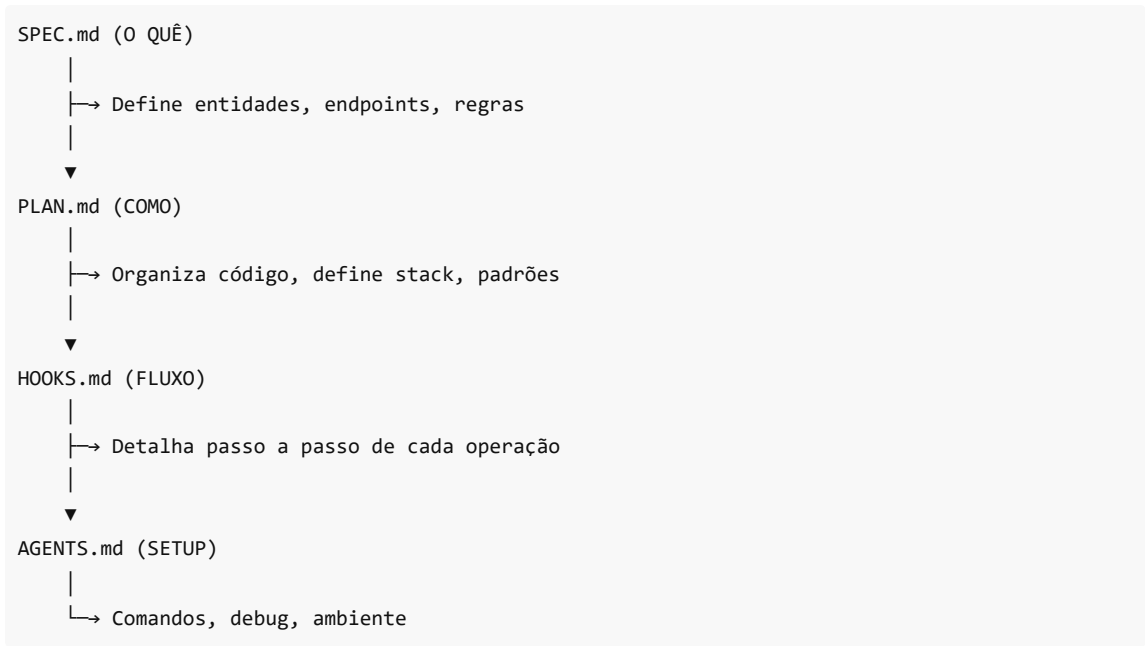
Para quem	Benefício
Assistentes de IA (OpenCode, Claude, GPT)	Entender e codificar o projeto sem contexto adicional
Novos desenvolvedores	Ramp-up rápido em horas, não dias
Documentação viva	Mantém rastreabilidade entre arquitetura e código
Debug	HOOKS.md mostra fluxos esperados vs erros reais

Diagrama do Harness

HARNESS			
PLAN.md	SPEC.md	HOOKS.md	AGENTS.md
"Como está organizado"	"O que o sistema faz"	"Como funciona"	"Como rodar"
- Diretórios	- Entidades	- Fluxo login	- Setup
- Stack	- API endpoints	- Fluxo criar	- Comandos
- Padrões	- Regras	- Fluxo bulk	- Debug
- Configuração	- Permissões	- Fluxos erro	- Portas

2. ESTRUTURA DOS 4 DOCUMENTOS

2.1 Relacionamento entre documentos



2.2 Tamanho ideal

Documento	Linhas	Foco
PLAN.md	150-250	Técnico/arquitetura
SPEC.md	200-350	Funcional/regras
HOOKS.md	250-400	Sequências/detalhes
AGENTS.md	80-150	Operacional

3. PLAN.md - ARQUITETURA DO SISTEMA

3.1 Seções obrigatórias

```
# PLAN – [Nome do Sistema]

## 1. Estrutura de Diretórios

[Caminho raiz]/
├─ backend/
│   └─ src/main/java/com/[projeto]/
│       ├── config/           → Configurações (CORS, Security, ExceptionHandler)
│       ├── controller/       → REST endpoints
│       ├── dto/              → Data Transfer Objects
│       ├── model/            → Entities JPA
│       ├── repository/       → Spring Data JPA
│       └─ service/           → Regras de negócio
├─ frontend/
│   └─ src/app/
│       └─ components/        → Componentes standalone
```

		services/	→ Serviços HTTP
		models/	→ Interfaces TypeScript
		guards/	→ Auth guards
		interceptors/	→ HTTP interceptors
		[scripts/ ou docs/]	→ Utilitários

Stack Tecnológica

Camada	Tecnologia	Versão
Backend	Spring Boot	3.x
Frontend	Angular	21
Database	[H2/PostgreSQL/Oracle]	-
Testes BE	JUnit + MockMvc	5.x
Testes FE	Playwright	-

Fluxo de Comunicação

[Browser] ↔ [Angular :4200] ←HTTP JSON→ [Spring Boot :8080] ←JDBC→ [Database]

Padrões e Convenções

Autenticação

- [Descrever: JWT / Header customizado / Session]
- [Onde armazena: localStorage / cookies]

Headers HTTP

- Header: X-User-Id ou Authorization: Bearer
- [Como adicionar no frontend: interceptor]

Permissões

- [Hierarquia de roles]
- [Onde validar: service ou @PreAuthorize]

Componentes

- [Standalone ou NgModule]
- [Template: inline ou externo]

CSS

- [Framework: Tailwind / Bootstrap / puro]
- [Responsividade: breakpoints]

Configuração de Desenvolvimento

Propriedade	Valor
Servidor backend	localhost:8080
H2 Console	/h2-console

JDBC URL	jdbc:h2:mem:[database]
Frontend dev	ng serve --open
CORS allowed	http://localhost:4200

Dependências Principais

Backend (pom.xml): spring-boot-starter-web, spring-boot-starter-data-jpa, h2, validation-api

Frontend (package.json): @angular/core@21, @angular/router, zone.js

4. SPEC.md - ESPECIFICAÇÃO FUNCIONAL

4.1 Seções obrigatórias

```
# SPEC – [Nome do Sistema]

## 1. Visão Geral

| Item | Valor |
|-----|-----|
| Propósito | [Descrever em 1 frase] |
| Frontend | Angular 21 standalone |
| Backend | Spring Boot 3.x |
| Database | H2 (dev) / [Prod] (prod) |
| Autenticação | [Tipo] |
```

Modelo de Dados

```
[ENTIDADE_1]
├ id: Long (PK, auto)
├ nome: String(100) [NOT NULL]
├ email: String(100) [UNIQUE]
├ status: [ENUM: ATIVO | INATIVO]
├ data_criacao: LocalDateTime
└ [relacionamentos: FK_entidade2_id]
```

Enums

[NomeEnum]: valores em ordem hierárquica se houver (ex: ADMIN > MANAGER > USER)

API Endpoints

Autenticação

Método	Rota	Auth	Descrição
POST	/api/auth/login	—	Login (email + senha) → UsuarioDTO

Recurso Principal

Método	Rota	Header	Role	Descrição
GET	/api/recursos	X-User-Id	Qualquer	Listar (com filtros)
GET	/api/recursos/{id}	—	—	Buscar por ID
POST	/api/recursos	X-User-Id	ADMIN	Criar
PUT	/api/recursos/{id}	X-User-Id	ADMIN	Atualizar
DELETE	/api/recursos/{id}	X-User-Id	ADMIN	Excluir

Regras de Permissão

Ação	ADMIN	MANAGER	USER
Login	✓	✓	✓
Listar	✓	✓	✓
Criar	✓	✓	✗
Editar	✓	✓	✗
Excluir	✓	✗	✗

Erro de permissão → HTTP 403 `{"erro": "Acesso negado: role X necessaria"}`

Regras de Negócio

1. **[Regra 1]:** descrição detalhada
2. **[Regra 2]:** incluindo validações
3. **[Regra 3]:** triggers automáticos

Frontend — Rotas

Rota	Componente	Descrição
/login	LoginComponent	Tela de login
/	ListComponent	Listagem principal
/novo	FormComponent	Criar registro
/editar/:id	FormComponent	Editar registro

Tratamento de Erros

Situação	HTTP	Body
Acesso negado	403	<code>{"erro": "..."} </code>
Violação unique	400	<code>{"erro": "Email ja cadastrado."} </code>
Erro validação	400	<code>{"erro": "campo: mensagem"} </code>
Não encontrado	400	<code>{"erro": "Recurso nao encontrado: {id}"} </code>

5. HOOKS.md - FLUXOS DETALHADOS

5.1 Seções obrigatórias

```
# HOOKS – Fluxos e Operações do [Nome do Sistema]
```

Fluxo de Autenticação

```
Usuário abre /login
→ LoginComponent.render(): form com email + senha
→ Usuário preenche e submit
→ LoginComponent.onSubmit()
→ AuthService.login(email, senha)
→ POST /api/auth/login {email, senha}
← 200 {id, nome, email, role}
→ salva em localStorage.setItem('usuarioLogado', JSON.stringify(response))
→ [Se usar window.location.href ou router.navigate]
→ AuthGuard: verifica localStorage
```

Fluxo de Criação de [Recurso]

```
Usuário clica "Novo"
→ router.navigate(['/novo'])
→ FormComponent.render()
→ Carrega dados auxiliares (ex: categorias, usuários)
→ Form com campos validados
→ Submit
→ [Aplica transformações: uppercase, trim]
→ Service.criar(dto)
→ POST /api/recursos com header X-User-Id
← 200 RecursoDTO
→ router.navigate([''])
```

Fluxo de [Ação Específica]

```
[Nome da ação]
[Responsável ou ROLE necessária]
→ [Evento disparador]
→ [Requisição HTTP]
← [Resposta esperada]
→ [Atualização UI: sem recarregar tudo]
```

Fluxo de Busca e Filtros

```
Ao carregar / (ListComponent)
→ ngOnInit(): carrega todos recursos
→ Aplica filtro ativo + ordenação
→ Renderiza grid
```

Filtros disponíveis:

"Todos" (padrão) → listaCompleta

"Meus recursos" → listaCompleta.filter(...)

"Atrasados" → listaCompleta.filter(...)

Busca:

→ searchTerm ligado a input

→ listaFiltrada.filter(...)

Ordenação padrão:

[Critério 1: urgência, data, etc]

[Critério 2]

Fluxo de Ações em Massa (se aplicável)

Seleção:

→ Checkbox em cada card

→ Checkbox "Selecionar todos"

→ Barra de ações aparece com ≥ 1 selecionado

[Ação em massa 1]:

→ Para cada item selecionado:

PATCH /api/recursos/{id}/[acao]

→ Aguarda todas Promises

→ Atualiza lista localmente (sem recarregar tudo)

[Ação em massa 2] (apenas ADMIN):

→ Confirma com window.confirm

→ Para cada item: DELETE /api/recursos/{id}

→ Remove do array local

Fluxo de Edição

FormComponent em modo edição

→ ngOnInit: id = ActivatedRoute.params.id

→ Carrega recurso via GET /api/recursos/{id}

→ Preenche form

→ Submit:

→ PUT /api/recursos/{id}

← 200 RecursoDTO

→ router.navigate(['/'])

Tratamento de Erros (Backend → Frontend)

Controller → Service

→ Se sem permissão: RuntimeException("Acesso negado: ...") → 403

→ Se não encontrado: RuntimeException("... nao encontrada") → 400

→ Se duplicado: DataIntegrityViolation → 400

```
GlobalExceptionHandler
@ExceptionHandler(RuntimeException.class)
→ mensagem "Acesso negado" → 403
→ default → 400

Frontend captura:
error(err) {
    this.mensagemErro = err.error?.erro || 'Erro desconhecido';
}
```

Ciclo de Vida dos Componentes

Componente Standalone

- constructor() → Injeta serviços (sem chamadas HTTP)
- ngOnInit() → Carrega dados (chamadas HTTP)
- ngOnDestroy() → Limpa subscriptions (se necessário)

AppComponent (topbar):

- Lê usuário do localStorage
- Expõe nome, role para template
- logout(): remove localStorage + window.location.href = '/login'

Inicialização do Banco (DataInitializer)

```
ApplicationRunner.run()
→ Se COUNT(Entidade) == 0:
    → Cria usuários padrão (ADMIN, USER)
    → Cria categorias/tipos padrão
    → Cria dados de exemplo (N registros)
```

Erros Comuns e Debug

Sintoma	Causa	Solução
403 ao criar	Sessão expirou ou role insuficiente	Relogar como ADMIN
"Email ja cadastrado"	Duplicado	Usar email diferente
Login não redireciona	router.navigate em vez de window.location	Usar setTimeout + window.location

6. AGENTS.md - SETUP PARA IA

6.1 Seções obrigatórias

```
# AGENTS.md - [Nome do Sistema]

**Para uso com assistentes de IA (OpenCode, Claude, Cursor, Copilot)**
```

Stack Real

Componente	Tecnologia	Versão
Frontend	Angular	21 (standalone)
Backend	Spring Boot	3.x
Java	OpenJDK	17+
Database	H2 (dev)	mem mode
Testes BE	JUnit + MockMvc	5.x
Testes FE	Playwright	-

Estrutura de Pastas (Resumida)

```

backend/src/main/java/com/[projeto]/
├─ config/ → Configurações
├─ controller/ → REST endpoints
├─ dto/ → Data Transfer Objects
├─ model/ → Entities
├─ repository/ → JPA
└─ service/ → Regras de negócio

frontend/src/app/
├─ components/ → Standalone
├─ services/ → HTTP
├─ models/ → Interfaces
├─ guards/ → Auth
└─ interceptors/ → HTTP interceptors

```

Regras de Setup (OBRIGATÓRIO)

NUNCA rode comandos sem verificar:

1. Se `node_modules` existe → usar `npm ci` em vez de `npm install`
2. Se `target/` existe → usar `mvnw clean compile` antes de `run`
3. Para primeira execução, rode `scripts/setup.sh` UMA vez

Proibido para o agente:

- Rodar `npm install` em projetos grandes (muito lento)
- Indexar arquivos dentro de `node_modules/` ou `target/`

Como rodar

Backend

```

cd backend
mvnw spring-boot:run
# API: http://localhost:8080/api
# H2: http://localhost:8080/h2-console

```

Frontend

```
cd frontend
ng serve --open
# App: http://localhost:4200
```

Ambos simultaneamente (2 terminais)

Testes

```
# Backend
cd backend && mvnw test

# Frontend E2E (servidor precisa estar rodando)
cd frontend && npm run e2e
```

Acessos

URL	Descrição
http://localhost:4200	Frontend
http://localhost:8080/api	API REST
http://localhost:8080/api/health	Health Check
http://localhost:8080/h2-console	H2 Console

Credenciais Padrão (se aplicável)

Role	Email	Senha
ADMIN	admin@[dominio].com	admin123
USER	user@[dominio].com	123456

Comandos úteis para debug

Problema	Comando
H2 não conecta	JDBC: jdbc:h2:mem:[database];MODE=Oracle
Reset BD	rm -rf backend/target/ + mvnw clean
Porta 8080 em uso (Windows)	netstat -ano findstr :8080
Porta 8080 em uso (Linux/Mac)	lsof -i :8080
Limpar cache Angular	rm -rf frontend/.angular/cache
Ver logs backend	tail -f backend/logs/application.log (se configurado)

APIs Principais (resumo)

- GET /api/recursos - Listar
- GET /api/recursos/{id} - Buscar
- POST /api/recursos - Criar (ADMIN)
- PUT /api/recursos/{id} - Atualizar (ADMIN)
- DELETE /api/recursos/{id} - Excluir (ADMIN)

7. CHECKLIST DE CRIAÇÃO

7.1 Para cada documento

Documento	Verificações
PLAN.md	Estrutura de diretórios completa Stack com versões específicas Padrões de código definidos Configuração de desenvolvimento
SPEC.md	Modelo de dados completo Todos endpoints documentados Matriz de permissões Regras de negócio listadas
HOOKS.md	Fluxo de autenticação detalhado CRUD completo Ações em massa (se houver) Tratamento de erros Erros comuns e soluções
AGENTS.md	Stack resumida Comandos de setup Acessos e credenciais Comandos de debug

7.2 Consistência entre documentos

- ☐ Mesmo nome de entidades em todos os documentos
- ☐ Mesmas roles/permissões em SPEC e PLAN
- ☐ Rotas do SPEC correspondem aos fluxos do HOOKS
- ☐ Comandos do AGENTS funcionam (testados)

8. TEMPLATES PRONTOS

Template minimalista para início rápido

```
# PLAN.md
- Estrutura: backend/ (config, controller, dto, model, repository, service) + frontend/ (components, services, models)
- Stack: Spring Boot 3 + Angular 21 + H2
```

- Auth: Header X-User-Id (localStorage)
- CSS: Grid responsivo (4→3→2→1 colunas)

SPEC.md

Entidades: Usuario(id, nome, email, role), Recurso(id, nome, status)

Roles: ADMIN > MANAGER > USER

Endpoints: CRUD padrão + /api/auth/login

Regras: Nome obrigatório, email único

HOOKS.md

Fluxo: Login → localStorage → Guard → List

Criar: Form → Service → POST → Navigate

Erros: 403 (acesso), 400 (validação), 404 (não encontrado)

AGENTS.md

Setup: cd backend && mvnw spring-boot:run | cd frontend && ng serve

Testes: mvnw test (BE) | npm run e2e (FE)

Debug: H2 (jdbc:h2:mem:test), Kill porta 8080 (lsof -i :8080)

COMO USAR ESTE GUIA

Passo 1: Identifique seu projeto

- Qual o domínio? (tarefas, produtos, usuários, etc)
- Quantas entidades principais? (comece com 1-2)
- Precisa de autenticação?

Passo 2: Copie os templates

- Use as seções 3, 4, 5, 6 como base
- Adapte nomes e estruturas

Passo 3: Preencha iterativamente

1. Comece com **SPEC.md** (o que o sistema faz)
2. Depois **PLAN.md** (como organizar)
3. Depois **HOOKS.md** (detalhar fluxos)
4. Por fim **AGENTS.md** (setup operacional)

Passo 4: Valide

- Teste os comandos do AGENTS.md
- Verifique se um desenvolvedor novo consegue rodar o projeto só com os 4 arquivos

Versão: 1.0 **Baseado no** Harness do TaskFlow **Tamanho ideal:** 4 arquivos, ~800 linhas no total